SCANTE · PLANO DE PRODUTO

Plano de implementação

Paridade funcional com Ivanti Velocity

Escopo técnico e sequência de desenvolvimento para transformar os recursos existentes do ScanTE em uma plataforma de emulação e operação logística configurável, segura e pronta para frota.

Data: 22 de outubro de 2026
Estado: plano inicial para desenvolvimento
Base: código ScanTE e documentação pública da Ivanti

Direção do produto

O ScanTE deve atender operações logísticas que hoje usam terminais Telnet ou aplicações Web em coletores. A meta é cobrir os recursos úteis do Velocity — conexão, emulação, configuração central, adaptação de telas, leitores, automação e continuidade — sem copiar código, formatos binários, marcas ou ativos proprietários da Ivanti.

"Paridade" será medida por casos de uso aprovados e testados em hosts e coletores representativos. Uma caixa marcada como existente abaixo indica presença no repositório ou na documentação local; não significa que a função já passou por homologação de produção.

O que já existe no ScanTE

Área	Presença observada	Situação para este plano
Cliente Telnet Android, sessões e emulador VT	`EmuladorTelnet/`	Base existente; fechar matriz de compatibilidade e testes de host.
Conexões simultâneas e troca de sessão	Cliente documenta múltiplas sessões; `docs/EM-ANDAMENTO.md` registra limite atual maior que o manual antigo	Atualizar documentação e confirmar limite/configuração real.
SSH, TLS, proxy e opções de comunicação	`TelnetClient.kt`, telas de configuração e manual	Implementações existentes; revisar segurança e cobertura de protocolo.
Leitura de código de barras e teclados configuráveis	Cliente Android e `scante-browser/`	Base existente; inventariar compatibilidade por fabricante/modelo.
Browser industrial e atalhos	`scante-browser/`	Base existente; validar requisitos de navegador corporativo e gerenciamento.
Gravador de telas Telnet	`ScreenRecorder.kt` e `docs/GRAVADOR-DE-TELAS.md`	Captura existente; editor e formato de configuração renderizável ainda faltam.
PoC de tela/formulário reformulado	Branch atual e `docs/homologacao/centauro/`	Demonstração específica; substituir por motor genérico orientado a perfis.
Console/configuração e administração	`scante-config/`, `scante-admin/`	Componentes existentes; definir qual será a ferramenta principal para projetos e publicação.
Relay Windows	`scante-relay/`	Existe reconexão TCP; não considerar equivalente a sessão persistente até validar os casos abaixo.
Licenças, distribuição e monitoramento	`scante-admin/`, `scante-monitor/`	Ecossistema existente; ampliar para configuração e suporte de frota.

Lacunas principais em relação ao produto documentado

1. **Tipos de host e protocolos:** a documentação do Velocity inclui VT100/VT220, IBM 5250/3270, Web e Oracle SIM. O ScanTE é hoje focado em VT/Telnet e possui um browser separado; não há evidência de emulação 5250/3270 ou Oracle SIM.
2. **Modernização de telas:** o gravador já captura telas, mas falta um editor para identificar elementos e um runtime genérico que renderize as configurações publicadas. A PoC atual é específica.

3. **Provisionamento de frota:** falta consolidar perfis por host, configurações globais, implantação versionada, QR/Scan-to-Config e política de atualização/rollback em um fluxo coerente.
4. **Automação e formulários:** faltam um mecanismo seguro de regras/scripts e formulários empresariais publicados como configuração.
5. **Continuidade de sessão:** o Relay mantém a conexão do lado do coletor aberta enquanto tenta abrir outro socket TCP ao host. Isso não prova que o estado da aplicação Telnet original sobreviva. O Relay também acumula e reenvia bytes digitados durante a queda; essa repetição pode executar uma ação duplicada ou fora de contexto. Não anunciar "continua do ponto onde parou" até desenhar e homologar a semântica com o ERP.
6. **Segurança de TLS:** `TelnetClient.buildSslContext()` contém um `TrustManager` que aceita qualquer certificado. O transporte pode estar cifrado sem autenticar o servidor. Corrigir antes de tratar TLS como seguro para produção.
7. **Voz e operação assistida:** não foi encontrada integração equivalente a fala para texto/texto para fala, nem um fluxo unificado de diagnóstico de sessão e captura de logs para suporte.
8. **Documentação de produto:** `README.md`, `MANUAL_ScanTE.md`, `EmuladorTelnet/PROJECT_STATUS.md` e `docs/EM-ANDAMENTO.md` descrevem fases diferentes do produto. Este plano deve ser a referência para a evolução de paridade; os outros documentos precisam ser reconciliados em tarefa própria.

Arquitetura-alvo

Manter os aplicativos e serviços existentes enquanto suas responsabilidades ficam explícitas. Evoluir por interfaces e formatos versionados, evitando uma reescrita ampla antes de validar o núcleo.

```
ScanTE Android
  ├─ Runtime de sessão
  │   ├─ Transportes: Telnet, Telnet sobre TLS, SSH
  │   ├─ Negociação Telnet e adaptadores de protocolo/emulação
  │   ├─ Estado de sessão, reconexão e diagnóstico
  │   └─ API de entrada: teclado, leitor e voz
  ├─ Motor de terminal
  │   ├─ Modelo de tela/cursor/atributos
  │   ├─ Renderizador nativo e renderizador de formulários
  │   └─ Compatibilidade VT e, em etapa própria, IBM 5250/3270
  └─ Aplicador de configuração
      ├─ Perfis, teclados, templates, regras e formulários
      ├─ Pacotes ScanTE assinados, versionados e reversíveis
      └─ Importação, QR e atualização gerenciada

ScanTE Studio/Admin
  ├─ Cadastro de hosts e configurações globais
  ├─ Editor de telas alimentado por gravações autorizadas
  ├─ Editor de formulários e regras com pré-visualização
  └─ Publicação, versões, grupos de dispositivos e auditoria

ScanTE Relay
  ├─ Túnel e observabilidade
  └─ Continuidade somente com semântica de sessão comprovada
```

Decisões de projeto propostas

- Usar um formato ScanTE próprio, declarativo, versionado e validado por schema. Não importar nem tentar reproduzir `.wldep`, `.wlxgp` ou formatos internos do Velocity.
- Manter transporte, protocolo de emulação, estado da sessão e renderização separados. Um browser Web não deve ser forçado a passar pelo motor de terminal.
- Começar regras com ações declarativas limitadas. Se scripts JavaScript forem necessários para cobrir casos reais, executá-los em sandbox sem acesso arbitrário a arquivos, rede ou APIs Android.

- Assinar pacotes de configuração e validar assinatura, versão, empresa e compatibilidade antes de aplicar. Permitir rollback para a última versão válida.
- Separar segredos de configuração exportável; nunca colocar senha de host, chave privada ou token em QR/pacote de distribuição em texto simples.
- TLS deve validar cadeia e hostname por padrão. Certificados privados precisam de uma forma explícita de provisionar CA/certificado confiável, sem `TrustManager` permissivo.

Plano por fases

Fase 0 — Inventário confiável e segurança do núcleo

- ☐ Criar matriz real de recursos por módulo: implementado, parcialmente implementado, ausente e validado em dispositivo.
- ☐ Conciliar limites de sessões, requisitos Android e funções do manual com o código atual.
- ☐ Remover a confiança irrestrita em certificados TLS; definir validação de hostname e suporte a CA corporativa.
- ☐ Revisar armazenamento de credenciais, chaves SSH, logs e exportações de sessão.
- ☐ Documentar a máquina de estados de conexão e reconexão do Relay, inclusive descarte/reenvio de dados.
- ☐ Definir dispositivos-alvo e hosts de homologação (VT/Protheus, IBM, navegador Web e leitores de cada fabricante).

Saída: inventário aprovado, riscos de segurança tratados ou bloqueando claramente o uso, matriz de compatibilidade e ambiente de homologação reproduzível.

Fase 1 — Núcleo de terminal e compatibilidade

- ☐ Formalizar interface de transporte e encapsular Telnet, TLS e SSH sem misturar parsing de tela com I/O.
- ☐ Fechar suporte Telnet necessário aos hosts-alvo: IAC, BINARY, ECHO, SGA, TTYPE, NAWS, subnegociação e sequências de encerramento.
- ☐ Definir suporte real por terminal (VT100, VT220, XTERM), tamanho de tela, charset/codepage e comportamento de teclas.
- ☐ Criar catálogo de sequências suportadas e comportamento previsível para as não suportadas.
- ☐ Homologar sessões longas, rede intermitente, retomada pelo operador, múltiplas sessões e dispositivos de baixa memória.
- ☐ Tratar TN5250/TN3270 como adaptadores/protocolos próprios, com estudo de viabilidade e host IBM de teste; não chamar um terminal VT de emulação IBM.

Saída: matriz por host/terminal aprovada com evidência de funcionamento e limites documentados.

Fase 2 — Configuração central e provisionamento

- ☐ Definir schema `ScanTE Project` versionado para perfis, ajustes globais, teclado, leitor, telas, regras e formulários.
- ☐ Separar configuração por perfil de host das opções globais hoje compartilhadas, preservando compatibilidade com configurações existentes.
- ☐ Implementar geração, assinatura, importação, validação, atualização incremental e rollback do pacote.
- ☐ Completar fluxo de provisioning por QR/Scan-to-Config sem incluir segredos no código de barras.

- ☐ Integrar distribuição e estado de implantação ao Admin/Config existente; definir grupos de dispositivos e trilha de auditoria.
- ☐ Definir política de MDM: distribuição do APK, configuração gerenciada e compatibilidade com StageNow/Intune ou equivalentes, conforme os clientes-alvo.

Saída: TI consegue preparar um coletor novo sem editar manualmente o perfil em cada dispositivo, e reverter uma configuração defeituosa.

Fase 3 — Editor de telas e runtime de modernização

- ☐ Evoluir o gravador para associar cada gravação ao host/perfil e capturar telas/estados sem guardar valores digitados sensíveis.
- ☐ Criar editor visual de grade com seleção de rótulos, campos de entrada, botões, mensagens e regiões variáveis.
- ☐ Criar identificadores robustos de tela que tolerem dados variáveis; não depender somente de uma frase exata.
- ☐ Criar pré-visualização com exemplos gravados e simulação de navegação/teclas.
- ☐ Implementar runtime genérico Android que renderize formulários e traduza ações para entradas do host.
- ☐ Ter fallback para a tela nativa quando não houver regra/template correspondente e permitir retorno à tela nativa conforme política.
- ☐ Implementar versão/publicação e associação do template ao perfil; substituir a PoC específica por configuração carregada do pacote.

Saída: fluxo completo gravação autorizada → edição → preview → publicação → aplicação no coletor, com fallback comprovado.

Fase 4 — Leitores, automação, formulários e voz

- ☐ Definir API comum de entrada para leitor integrado, wedge/teclado e câmera, com configuração por fabricante.
- ☐ Permitir validação, normalização, prefixo/sufixo e roteamento de leitura por campo/formulário.
- ☐ Implementar formulários de processo com campos, validação, listas, mensagens, botões e envio controlado ao host/API.
- ☐ Entregar automações como regras declarativas primeiro; adicionar JavaScript sandbox somente se a matriz de casos exigir.
- ☐ Adicionar leitura de tela e fala para texto, com idioma, acionamento, cancelamento e tratamento para ambiente ruidoso.
- ☐ Preservar teclado de função, atalhos e fluxo de trabalho sem bloquear o scanner físico.

Saída: fluxo de recebimento/picking configurável sem mudança no ERP, homologado com leitor e usuário operacional.

Fase 5 — Continuidade, suporte e operação de frota

- ☐ Especificar garantias do Relay: o que é mantido, o que é retomado e quando exigir novo login.
- ☐ Remover replay cego de teclas/scans após perda do socket; usar confirmação/protocolo de aplicação ou descartar com mensagem segura.
- ☐ Investigar suporte de sessão persistente com host/ERP, proxy de sessão compatível ou estratégia explícita de reconexão por transação.
- ☐ Adicionar telemetria local opt-in, logs redigidos e pacote de diagnóstico exportável sem credenciais/dados pessoais.

- ☐ Adicionar console de sessões/dispositivos e suporte remoto sujeito a autorização e auditoria.
- ☐ Medir reconexão, consumo, bateria e recuperação de falhas em rede real.

Saída: suporte consegue diagnosticar incidentes e o produto descreve corretamente o comportamento após cada classe de falha.

Fase 6 — Emulações adicionais e expansão de plataforma

- ☐ Priorizar IBM 5250/3270 a partir de demanda e host de homologação disponível.
- ☐ Definir se o Browser industrial permanece app separado ou vira modo do ScanTE, mantendo cookies, isolamento e política de segurança claros.
- ☐ Avaliar cliente Windows e iOS por demanda comercial; compartilhar schema e regras, não presumir reutilização de UI Android.
- ☐ Criar estratégia de compatibilidade de versões e suporte de dispositivos de ciclo longo.

Saída: decisão explícita de produto e plano separado para cada plataforma/protocolo adicional.

Ordem de execução recomendada

1. Fase 0: corrigir a base de segurança e validar o que já funciona.
2. Fase 1: estabilizar protocolos, terminais e sessões usadas pelos clientes atuais.
3. Fase 2: fechar o ciclo de configuração e distribuição.
4. Fase 3: transformar gravador e PoC em editor/runtime genéricos.
5. Fase 4: completar automação, leitores e voz conforme os fluxos reais.
6. Fase 5: prometer continuidade somente depois de validar o estado da aplicação host.
7. Fase 6: ampliar protocolos e plataformas mediante demanda e infraestrutura de homologação.

Critérios de pronto por recurso

Uma funcionalidade só passa a “pronta” quando:

- possui requisito e comportamento documentados;
- tem configuração versionada e migração definida quando necessário;
- funciona no dispositivo físico e no servidor/ERP de homologação definidos;
- cobre erro, cancelamento, queda de rede e retorno à operação;
- não expõe credenciais nem dados sensíveis em log, gravação, QR ou exportação;
- possui responsável, versão mínima e limitações documentadas;
- foi validada por TI e por usuário operacional quando altera o fluxo de trabalho.

Referências de comportamento do Velocity

- Google Play — Ivanti Velocity
- Introdução e tipos de host
- Console, projetos e Scan-to-Config
- Perfis de host
- Scripts JavaScript
- Instalação e plataformas

- Session Persistence Server